Cryptography Fundamentals

Security Fundamentals Assessment: core cryptography concepts and their role in protecting sensitive information.

1. Core Concepts

Concept	What it does	Goal it serves
Cryptography	The science of securing information using mathematical techniques	Confidentiality, integrity, authentication, non-repudiation
Encryption	Converts plaintext into unreadable ciphertext using an algorithm and a key	Confidentiality
Decryption	Reverses encryption to recover the plaintext (requires the correct key)	Authorized access
Hashing	One-way function producing a fixed-size digest from any input; it cannot be reversed	Integrity
Digital signature	Hash of a message signed with the sender's private key, verified with their public key	Authenticity, integrity, non-repudiation

Hashing is not encryption, because there is no key and no way to decrypt. Digital signatures prove who sent something and that it wasn't altered, but they do not hide its contents.

2. Symmetric vs. Asymmetric Encryption

	Symmetric	Asymmetric
Keys	One shared secret key	Key pair (public and private)
Speed	Very fast	Slow (100-1000x slower)

	Symmetric	Asymmetric
Best for	Bulk data (files, disks, database fields, streams)	Key exchange, signatures, identity
Main challenge	Securely distributing the key	Computational cost, trust in public keys (needs PKI/certificates)
Examples	AES, ChaCha20	RSA, ECC (ECDSA, ECDH), Ed25519

In practice, systems combine both. TLS uses asymmetric cryptography to authenticate the server and agree on a session key, then uses symmetric encryption (AES-GCM or ChaCha20) for the actual traffic. This is called **hybrid encryption**.

3. Common Algorithms

AES (Advanced Encryption Standard)

- A symmetric block cipher with 128-bit blocks and 128, 192, or 256-bit keys.
- AES-256 is the usual choice for high-sensitivity data.
- Always use an authenticated mode such as **GCM**. Avoid ECB, which leaks patterns, and unauthenticated CBC.
- Never reuse a nonce/IV with the same key in GCM.

RSA

- An asymmetric algorithm based on the difficulty of factoring large primes.
- Used for encryption, key transport, and signatures.
- Use at least **2048-bit** keys (3072+ recommended for long-term security) with OAEP padding for encryption and PSS for signatures.
- Vulnerable to future quantum computers (Shor's algorithm).

ECC (Elliptic Curve Cryptography)

- Based on the elliptic curve discrete logarithm problem.
- Gives equivalent security with much smaller keys. A 256-bit ECC key is roughly equal to a 3072-bit RSA key.
- Common uses: ECDSA and Ed25519 (signatures), ECDH/X25519 (key exchange).
- Faster and lighter, so it suits mobile, IoT, and TLS 1.3. It is also quantum-vulnerable.

SHA-256

- Part of the SHA-2 family, producing a 256-bit digest.
- Used for file integrity, certificates, digital signatures, blockchain, and HMAC.
- Collision-resistant and preimage-resistant. MD5 and SHA-1 are broken and should not be used for security.
- SHA-256 is **too fast for password storage**. Use a slow, salted algorithm such as **Argon2id, bcrypt, or scrypt** instead.

4. Choosing Techniques for Different Data

Data / Scenario	Recommended approach
Data at rest (databases, disks, backups)	AES-256-GCM, full-disk encryption, field-level encryption for sensitive columns, keys in a KMS/HSM
Data in transit (web, APIs)	TLS 1.2+ (preferably 1.3) with ECDHE key exchange and AES-GCM or ChaCha20-Poly1305
Passwords	Salted, slow hash (Argon2id, bcrypt, scrypt), never reversible encryption
File/message integrity	SHA-256 or SHA-3, or HMAC-SHA-256 if a shared secret is available
Software/code distribution	Digital signatures (RSA-PSS, ECDSA, Ed25519) plus published hashes
Email / end-to-end messaging	Hybrid encryption (e.g., Signal protocol, PGP/S/MIME)
Payment and healthcare data	AES-256, tokenization, strict key management (PCI DSS, HIPAA)
Tokens / API authentication	HMAC-SHA-256 or signed tokens (JWT with RS256/ES256)
Long-term secrets	Larger key sizes and a plan for post-quantum migration

5. Key Use Cases and Security Considerations

Use cases

Secure web browsing (HTTPS), VPNs, encrypted storage, authentication, code signing, blockchain and cryptocurrency, digital certificates, secure messaging.

Security considerations

- **Key management is the weakest link.** Generate keys with a cryptographically secure random number generator, store them in an HSM or KMS, rotate them regularly, and never hardcode them.
- **Don't roll your own crypto.** Use vetted libraries (libsodium, OpenSSL, Bouncy Castle).
- **Use authenticated encryption** so tampering is detected, not just hidden.
- **Avoid deprecated algorithms:** DES, 3DES, RC4, MD5, SHA-1, and RSA below 2048 bits.
- **Protect against implementation attacks** such as side channels, padding oracles, and nonce reuse.
- **Verify identity** with certificates and PKI, or encryption alone can be defeated by man-in-the-middle attacks.
- **Plan for quantum threats.** Quantum computers could break RSA and ECC. NIST has standardized post-quantum algorithms (ML-KEM, ML-DSA), and sensitive long-lived data may need hybrid or migrated schemes now ("harvest now, decrypt later").
- **Comply with standards:** NIST, FIPS 140-3, PCI DSS, GDPR, and HIPAA.

Quick Summary

- **Encrypt** to keep data secret, **hash** to detect changes, **sign** to prove origin.
- **AES** for bulk data, **RSA/ECC** for key exchange and signatures, **SHA-256** for integrity.
- Strong algorithms fail when keys are mishandled, so key management matters as much as algorithm choice.